12. Projektion II - Gruppierung, Aggregation

Disclaimer:

Dieses Material ist lizensiert.

Lizenz: CC BY-NC-ND 4.0 (wenn nicht anders angegeben)

- **Attribution (BY)**
- **Non Commercial (NC)**
- **No Derivatives (ND)**
- **No Warranty**
- **No Liability**

Einzelne Beispiele oder seperat geteilte Dokumente könnten einer anderen Lizenz unterliegen.

Autor: © Korobov Viktor, 2026

Netzwerke:

- LinkedIn
- GitHub
- YouTube
- LeetCode
- Steam

WIP:

Dieses Kursmaterial ist ein lebendiges Dokument und wird mit einem KI Assisstenten gewartet.

Letzter Stand: 10.09.26.

Gebt aufmerksam Acht auf das Gesagte!

Unterstützung:

Bei Anmerkung zum Material kontakiert mich gerne unter:

- viktor.korobov@edu.korobov-solutions.de

Unterstützt mich mit Feedback unter:

- Fragebogen

Gruppierung, Aggregation

Aggregation

Aggregation bezeichnet den Vorgang, bei dem mehrere Zeilen einer Tabelle zu einem einzigen Ergebniswert zusammengefasst werden. Dies geschieht durch Anwendung sogenannter Aggregatfunktionen auf Spaltenwerte.

Aggregation ist hilfreich, wenn man aus einer großen Datenmenge Zusammenfassungen oder statistische Kennzahlen ableiten möchte, z. B.:

- Wie viele Einträge gibt es? → `COUNT()`
- Wie hoch ist der Umsatz insgesamt? → `SUM()`
- Wie hoch ist das durchschnittliche Gehalt? → `AVG()`

Wenn keine Gruppierung (`GROUP BY`) verwendet wird, wird die Aggregation über alle Zeilen der Tabelle berechnet:


```sql
SELECT COUNT(*) FROM bestellungen;
```

- Ergebnis: Die Gesamtanzahl der Bestellungen

Wichtige Aggregatfunktionen:

Funktion	Beschreibung	Rückgabewert	Besonderheiten
`COUNT(*)`	Zählt alle Zeilen	Ganzzahl	Zählt auch `NULL` -Werte
`COUNT(spalte)`	Zählt nur nicht-NULL-Werte in einer Spalte	Ganzzahl	`NULL` -Werte werden ignoriert
`SUM(spalte)`	Summiert alle Werte einer numerischen Spalte	Zahl	Nur numerisch verwendbar, `NULL` ignoriert
`AVG(spalte)`	Berechnet den Durchschnitt der Werte einer Spalte	Kommazahl (FLOAT)	Nur numerisch, `NULL` ignoriert
`MIN(spalte)`	Gibt den kleinsten Wert zurück	Datentyp der Spalte	Vergleichbar mit `ORDER BY ... ASC LIMIT 1`
`MAX(spalte)`	Gibt den größten Wert zurück	Datentyp der Spalte	Vergleichbar mit `ORDER BY ... DESC LIMIT 1`

Gruppierung

Unter Gruppierung versteht man das Zusammenfassen von Datensätzen anhand gleicher Werte in einer oder mehreren Spalten. Technisch erfolgt dies mithilfe des Schlüsselworts `GROUP BY` .

Die Gruppierung bildet dabei die Grundlage dafür, dass Aggregatfunktionen nicht auf die gesamte Tabelle, sondern getrennt pro Gruppe angewendet werden.

Gruppierung ermöglicht es, statistische Auswertungen pro Kategorie durchzuführen, z. B.:

- Anzahl Bestellungen pro Kunde
- Durchschnittlicher Umsatz pro Region
- Höchstes Gehalt pro Abteilung

Ablauf der Gruppierung (konzeptionell):

1. SQL verarbeitet zunächst alle Zeilen der Tabelle.
2. Es bildet Gruppen basierend auf den angegebenen Gruppierungsspalten.
3. Auf jede Gruppe wird eine oder mehrere Aggregatfunktionen angewendet.
4. Das Ergebnis enthält eine Zeile pro Gruppe, mit den berechneten Werten.

Beispiel:

```sql
SELECT kunde_id, COUNT(*) AS anzahl_bestellungen
FROM bestellungen
GROUP BY kunde_id;
```

- Ergebnis: Eine Zeile pro Kunde, mit Anzahl seiner Bestellungen

```
+----------+----------------------+
| kunde_id | anzahl_bestellungen |
+----------+----------------------+
| 101      | 3                    |
| 102      | 7                    |
| 103      | 1                    |
+----------+----------------------+
```

Ausgabe: Eine Zeile pro Kunde, mit Anzahl seiner Bestellungen

---

Gruppenfilter - `Having`

Das Schlüsselwort `HAVING` dient in SQL dazu, **Gruppenergebnisse nach der Aggregation zu filtern**. Es erfüllt eine ähnliche Rolle wie `WHERE`, wird jedoch **nach** der Anwendung von `GROUP BY` und Aggregatfunktionen verwendet.

Unterschied zwischen `WHERE` und `HAVING`:

Kriterium	`WHERE`	`HAVING`
Filtert vor der Gruppierung	✅ Ja	❌ Nein
Filtert nach der Gruppierung	❌ Nein	✅ Ja
Zugriff auf Aggregatfunktionen	❌ nicht erlaubt	✅ vollständig erlaubt
Anwendungsfall	Einzelzeilen selektieren	Gruppenergebnisse filtern

Beispiel (Aggregat in `WHERE`):

```sql
SELECT kunde_id, COUNT(*)
FROM bestellungen
WHERE COUNT(*) > 2 -- ❌ Nicht erlaubt!
GROUP BY kunde_id;
```

- Fehlermeldung, da `COUNT(*)` in `WHERE` nicht zulässig ist

Beispiel (Aggregat in `HAVING`):

```sql
SELECT kunde_id, COUNT(*)
FROM bestellungen
GROUP BY kunde_id
HAVING COUNT(*) > 2;
```

- Zeigt nur Kunden mit mehr als 2 Bestellungen

Beispiel (`WHERE` und `HAVING` in Tandem):

```sql
SELECT kunde_id, COUNT(*) AS kunden_bestellungen
FROM bestellungen
WHERE bezahl_status = 'bezahlt'          -- Filtert einzelne Zeilen
GROUP BY kunde_id
HAVING kunden_bestellungen >= 5;            -- Filtert Gruppen
```

Aufgaben

> **Tipp:**
>
> Viele Wege führen zum Ziel! Eure Ergebnisse könnten anders aussehen und dennoch richtig sein.
>
> Die "*Musikschule*"-Datenbank ist außerdem lebendig. Darum könnten Daten aus den Beispielen geringfügig von euren abweichen.

1. Wie viele Teilnehmer sind jeweils in einem Kurs angemeldet?

Zielergebnis (in-etwa):

Kurs_ID	teilnehmeranzahl
1	2
2	3
3	4
4	2
5	2

2. In welchen Räumen finden mehr als ein Kurs statt?

Zielergebnis (in-etwa):

Raum_ID	kursanzahl
5	2

3. Wie viele Teilnehmer nehmen insgesamt an Kursen in jedem Raum teil?

Zielergebnis (in-etwa):

Raum_ID	teilnehmeranzahl
1	2
2	3
3	2
5	6

4. Wie viele Teilnehmer hat jeder Lehrer über alle seine Kurse hinweg unterrichtet?

Zielergebnis (in-etwa):

Lehrer_ID	teilnehmeranzahl
1	7
2	4
3	2

5. Wie groß ist die durchschnittliche Kapazität der Räume, in denen jeder Lehrer unterrichtet?

Zielergebnis (in-etwa):

←T→		Lehrer_ID	durchschnittskapazität
☐	Edit Copy Delete	1	15.6667
☐	Edit Copy Delete	2	15.5000
☐	Edit Copy Delete	3	8.0000

6. Wie viele Lehrer unterrichten in jedem Fachgebiet?

Zielergebnis (in-etwa):

Fachgebiet_ID	lehreranzahl
1	1
2	1
3	2
4	1
5	1

7. Wie viele Teilnehmer wurden in jedem Jahr geboren? (nutzen Sie `YEAR()`)

Zielergebnis (in-etwa):

geburtsjahr	teilnehmeranzahl
2007	2
2008	3
2009	2
2010	2
2011	1

8. Welches Teilnehmeralter war das höchste in einem bestimmten Raum?

Zielergebnis (in-etwa):

			Raum_ID	MaxAlter
☐	Edit	Copy	Delete	1	17.4932
☐	Edit	Copy	Delete	2	17.4932
☐	Edit	Copy	Delete	3	16.4247
☐	Edit	Copy	Delete	5	18.8329

9. Wie groß ist die Gesamtkapazität aller Räume?

Zielergebnis (in-etwa):

gesamtkapazität
61

10. Wie viele Personen wohnen jeweils in welcher Stadt (laut Primäradresse)?

Zielergebnis (in-etwa):

Stadt	personenzahl
Berlin	4
Bremen	4
Düsseldorf	4
Frankfurt	4
Hamburg	4
Köln	4
Leipzig	4
Mainz	4
München	4
Stuttgart	4

Zusatz:

- Wie hoch ist das durchschnittliche Alter der Teilnehmer je Kurs im aktuellen Jahr? (nutzen Sie `CURDATE()`, `YEAR()`)

Zielergebnis (in-etwa):

←T→				Kurs_ID	durchschnittsalter
☐	Edit	Copy	Delete	1	16.55480000
☐	Edit	Copy	Delete	2	15.91690000
☐	Edit	Copy	Delete	3	17.40615000
☐	Edit	Copy	Delete	4	15.67535000
☐	Edit	Copy	Delete	5	17.88355000

Danksagung:

Liebe Lernende, vielen Dank für die Aufmerksamkeit!

Liebe Veranstalter, vielen Dank für die Bühne!

Unterstützung:

Bei Anmerkung zum Material kontakiert mich gerne unter:

- viktor.korobov@edu.korobov-solutions.de

Unterstützt mich mit Feedback unter:

- Fragebogen